

As one could imagine this data being reoccurring and there being benefit to having uniformity and persistence in the data, I used this as an exercise to transform and create a SQLite database from the provided CSVs. I also created a notebook generate a PostGres database, which can be found on my github:

https://github.com/earlyann/Instagram_data_modeling

✓ Import Dependencies

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
# import psycpg2
from datetime import datetime, timedelta
import sqlite3
```

✓ Read in and clean data

```
from pathlib import Path
import pandas as pd

# Read all CSV files from current folder
data_dir = Path('.')
dfs = {}

for file_path in data_dir.glob('*.csv'):
    file_name = file_path.stem
    dfs[file_name] = pd.read_csv(file_path)

print(dfs.keys())

for key, df in dfs.items():
    globals()[f'df_{key}'] = df
```

```
dict_keys(['users', 'photo_tags', 'likes', 'tags', 'follows', 'photos', 'comm
```

```
# Check for nulls in each df
for key, df in dfs.items():
    print(f"Nulls in DataFrame {key}:")
    print(df.isnull().sum().sum())
```

```
Nulls in DataFrame users:
0
Nulls in DataFrame photo_tags:
0
```

```

Nulls in DataFrame likes:
0
Nulls in DataFrame tags:
0
Nulls in DataFrame follows:
0
Nulls in DataFrame photos:
0
Nulls in DataFrame comments:
0

```

```

# Replace spaces with underscores in the column names and make all lowercase
for key, df in dfs.items():
    df.columns = df.columns.str.replace(' ', '_')
    df.columns = df.columns.str.lower()

```

```

# Change yes/no columns to boolean values and data type
for key, df in dfs.items():
    for col in df.columns:
        # If columns only contain yes or no, change to boolean
        if df[col].isin(['yes', 'no']).all():
            df[col] = df[col].map({'yes': True, 'no': False})

```

```

# Convert the created columns to datetime data type
date_time_cols = ['created_dat', 'created_date', 'created_time', 'created_tim

for key, df in dfs.items():
    for col in date_time_cols:
        if col in df.columns:
            df[col] = pd.to_datetime(df[col], errors='coerce') # Coerce wil

```

```

/tmp/ipykernel_2577/1243572534.py:7: UserWarning: Parsing dates in %d-%m-%Y %
df[col] = pd.to_datetime(df[col], errors='coerce') # Coerce will return Na
/tmp/ipykernel_2577/1243572534.py:7: UserWarning: Parsing dates in %d-%m-%Y %
df[col] = pd.to_datetime(df[col], errors='coerce') # Coerce will return Na
/tmp/ipykernel_2577/1243572534.py:7: UserWarning: Parsing dates in %d-%m-%Y %
df[col] = pd.to_datetime(df[col], errors='coerce') # Coerce will return Na
/tmp/ipykernel_2577/1243572534.py:7: UserWarning: Parsing dates in %d-%m-%Y %
df[col] = pd.to_datetime(df[col], errors='coerce') # Coerce will return Na
/tmp/ipykernel_2577/1243572534.py:7: UserWarning: Parsing dates in %d-%m-%Y %
df[col] = pd.to_datetime(df[col], errors='coerce') # Coerce will return Na
/tmp/ipykernel_2577/1243572534.py:7: UserWarning: Parsing dates in %d-%m-%Y %
df[col] = pd.to_datetime(df[col], errors='coerce') # Coerce will return Na

```

```

# Create new id columns for the likes and follows dataframes as they will be
df_likes['like_id'] = (df_likes.index + 1).astype(int)
df_follows['follow_id'] = df_follows.index + 1

```

```

# Rename the columns in each dataframe to match the database schema
df_photos.rename(columns={'created_dat': 'created_date', 'id': 'photo_id'}, in
df_users.rename(columns={'id': 'user_id', 'private/public': 'private'}, inplac

```

```
df_tags.rename(columns={'id': 'tag_id'}, inplace=True)
df_comments.rename(columns={'user__id': 'user_id', 'id': 'comment_id', 'created': 'comment_created'}, inplace=True)
df_likes.rename(columns={'user_': 'user_id', 'photo': 'photo_id'}, inplace=True)
df_follows.rename(columns={'follower': 'follower_user_id', 'followee': 'followee_user_id'}, inplace=True)
```

✓ Create interaction dataframe from likes, comments, and follows dataframes

```
# Combine Likes, Comments, and Follows DataFrames into a single Interactions DataFrame
df_interactions = pd.concat([df_likes, df_comments, df_follows], ignore_index=True)

# Create interaction_id column
df_interactions['interaction_id'] = df_interactions.index + 1

# Add a new column to indicate the type of interaction ('like', 'comment', or 'follow')
df_interactions['interaction_type'] = pd.Series(['like'] * len(df_likes) + ['comment'] * len(df_comments) + ['follow'] * len(df_follows))

# Add an interaction date column (you can use 'created_time' column or any other date column)
df_interactions['interaction_date'] = df_interactions['created_time']

# List of columns to keep
keep_cols = ['interaction_id', 'interaction_type', 'interaction_date', 'user_id', 'photo_id', 'comment_id', 'like_id', 'follow_id']

# Remove columns that are not in the keep_cols list
df_interactions = df_interactions[[col for col in df_interactions.columns if col in keep_cols]]

# Make sure the id columns are integers even though they contain NaN values
id_cols = ['user_id', 'photo_id', 'comment_id', 'like_id', 'follow_id']
for col in id_cols:
    df_interactions[col] = df_interactions[col].astype('Int64')
```

✓ Create a SQLite Database

The database is designed in a star schema with the central facts table being the Interactions table and all others being the dimension tables.

```
import sqlite3

db_file = "insta_lite.db"

conn = sqlite3.connect(db_file)
cursor = conn.cursor()

print("Database created successfully!")
```

Database created successfully!

```
import os
print(os.getcwd())
```

/content

Double-click (or enter) to edit

```
# Define the database file
conn = sqlite3.connect(db_file)
import sqlite3

db_file = 'insta_lite.db'

# Connect to the database
conn = sqlite3.connect(db_file)
cursor = conn.cursor()

# Create the Users table
cursor.execute('''CREATE TABLE IF NOT EXISTS users (
                user_id INTEGER PRIMARY KEY,
                name TEXT,
                created_time DATE,
                private BOOLEAN,
                post_count INTEGER,
                verified_status BOOLEAN
            )''')

# Convert DataFrame to data in the Users table
df_users.to_sql('users', conn, if_exists='append', index=False)
print("Rows in Users table:", cursor.execute('SELECT COUNT(*) FROM users').f

# Create the Photos table
cursor.execute('''CREATE TABLE IF NOT EXISTS photos (
                photo_id INTEGER PRIMARY KEY,
                image_link TEXT,
                user_id INTEGER,
                created_date DATE,
                insta_filter_used BOOLEAN,
                photo_type TEXT,
                FOREIGN KEY (user_id) REFERENCES users(user_id)
            )''')

# Convert DataFrame to data in the Photos table
df_photos.to_sql('photos', conn, if_exists='append', index=False)
print("Rows in Photos table:", cursor.execute('SELECT COUNT(*) FROM photos')

# Create the Tags table
cursor.execute('''CREATE TABLE IF NOT EXISTS tags (
                tag_id INTEGER PRIMARY KEY,
                tag_text TEXT,
                created_time DATE,
                location TEXT
            )''')

# Convert DataFrame to data in the Tags table
df_tags.to_sql('tags', conn, if_exists='append', index=False)
print("Rows in Tags table:", cursor.execute('SELECT COUNT(*) FROM tags').fet
```

```

# Create the Comments table
cursor.execute('''CREATE TABLE IF NOT EXISTS comments (
    comment_id INTEGER PRIMARY KEY,
    user_id INTEGER,
    photo_id INTEGER,
    created_time DATETIME,
    posted_date TEXT,
    comment TEXT,
    emoji_used BOOLEAN,
    hashtags_used_count INTEGER,
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (photo_id) REFERENCES photos(photo_id)
)''')

# Convert DataFrame to data in the Comments table
df_comments.to_sql('comments', conn, if_exists='append', index=False)
print("Rows in Comments table:", cursor.execute('SELECT COUNT(*) FROM commen

# Create the Likes table
cursor.execute('''CREATE TABLE IF NOT EXISTS likes (
    like_id INTEGER PRIMARY KEY,
    user_id INTEGER,
    photo_id INTEGER,
    created_time DATE,
    following_or_not BOOLEAN,
    like_type TEXT,
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (photo_id) REFERENCES photos(photo_id)
)''')

# Convert DataFrame to data in the Likes table
df_likes.to_sql('likes', conn, if_exists='append', index=False)
print("Rows in Likes table:", cursor.execute('SELECT COUNT(*) FROM likes').f

# Create the Follows table
cursor.execute('''CREATE TABLE IF NOT EXISTS follows (
    follow_id INTEGER PRIMARY KEY,
    follower_user_id INTEGER,
    user_id INTEGER,
    created_time DATE,
    is_follower_active INTEGER,
    followee_acc_status TEXT,
    FOREIGN KEY (follower_user_id) REFERENCES users(user_id)
    FOREIGN KEY (user_id) REFERENCES users(user_id)
)''')

# Convert DataFrame to data in the Follows table
df_follows.to_sql('follows', conn, if_exists='append', index=False)
print("Rows in Follows table:", cursor.execute('SELECT COUNT(*) FROM follows

# Create the Interactions table
cursor.execute('''CREATE TABLE IF NOT EXISTS interactions (
    interaction_id INTEGER PRIMARY KEY,
    user_id INTEGER,
    photo_id INTEGER,

```

```

tag_id INTEGER,
comment_id INTEGER,
like_id INTEGER,
follow_id INTEGER,
interaction_date DATE,
interaction_type TEXT,
FOREIGN KEY (user_id) REFERENCES users(user_id),
FOREIGN KEY (photo_id) REFERENCES photos(photo_id),
FOREIGN KEY (tag_id) REFERENCES tags(tag_id),
FOREIGN KEY (comment_id) REFERENCES comments(comment_id),
FOREIGN KEY (like_id) REFERENCES likes(like_id),
FOREIGN KEY (follow_id) REFERENCES follows(follow_id)
)'''

```

```

# Convert DataFrame to data in the Interactions table
df_interactions.to_sql('interactions', conn, if_exists='append', index=False)
print("Rows in Interactions table:", cursor.execute('SELECT COUNT(*) FROM in

# Commit the changes and close the connection
conn.commit()
conn.close()

```

```

Rows in Users table: 100
Rows in Photos table: 257
Rows in Tags table: 21
Rows in Comments table: 7488
Rows in Likes table: 8782
Rows in Follows table: 7623
Rows in Interactions table: 23893

```

✓ Test Query the Database

```

import sqlite3

db_file = "insta_lite.db"

conn = sqlite3.connect(db_file)
cursor = conn.cursor()

print("Database created successfully!")

```

```
Database created successfully!
```

```

# Connect to the SQLite database
conn = sqlite3.connect(db_file)
import sqlite3

db_file = 'insta_lite.db'
cursor = conn.cursor()

# Query and print the # of photos in the photos table
cursor.execute("SELECT COUNT(*) FROM photos")
photo_count = cursor.fetchone()[0]

```

```
print(f"Number of photos in the photos table: {photo_count}")

# Query and print the # of users in the users table
cursor.execute("SELECT COUNT(*) FROM users")
user_count = cursor.fetchone()[0]
print(f"Number of users in the users table: {user_count}")

# Query the count of distinct tags in the tags table
cursor.execute("SELECT COUNT(DISTINCT tag_id) FROM tags")
tag_count = cursor.fetchone()[0]
print(f"Number of tags in the tags table: {tag_count}")

# Query and print the # of likes in the interactions table
cursor.execute("SELECT COUNT(*) FROM interactions WHERE interaction_type = 'like'")
like_count = cursor.fetchone()[0]
print(f"Number of likes in the interactions table: {like_count}")

# Query and print the # of comments in the interactions table
cursor.execute("SELECT COUNT(*) FROM interactions WHERE interaction_type = 'comment'")
comment_count = cursor.fetchone()[0]
print(f"Number of comments in the interactions table: {comment_count}")

# Query and print the # of follows in the interactions table
cursor.execute("SELECT COUNT(*) FROM interactions WHERE interaction_type = 'follow'")
follow_count = cursor.fetchone()[0]
print(f"Number of follows in the interactions table: {follow_count}")

# Close the connection
conn.close()
```

```
Number of photos in the photos table: 257
Number of users in the users table: 100
Number of tags in the tags table: 21
Number of likes in the interactions table: 8782
Number of comments in the interactions table: 7488
Number of follows in the interactions table: 7623
```

Business Insights

- A centralized database improves storage and management of social media data.
- Combining likes, comments, and follows into a single interactions table enables comprehensive user engagement analysis.
- The star schema design supports efficient querying and reporting.
- Data cleaning improves consistency and reduces redundancy.
- Structured databases provide a strong foundation for future analytics and dashboard development.

Conclusion

- Multiple Instagram datasets were integrated into a unified database structure.
- Data preprocessing and transformation improved data quality.
- A star schema was designed to support scalable analytical workloads.
- SQLite was used as a lightweight database solution for storing and querying data.
- The project demonstrates practical database design and data engineering concepts.